# Proxmox Storage DRS -- Operator Manual

**Installing, configuring and running it safely**

---

Bernd Zeimetz bernd@bzed.de

2026-09-07

Rendered from `docs/manual/*.md`, SHA-256 `735a0cc8fc0c631cebaa051c452931c89bac06b73ead22f2a5d0b94401ab219b` (`sha256sum --check docs/pve-storage-drs-manual.pdf.sha256` in the repository must succeed).

## Contents

### What you need

- **Proxmox VE 9.2**, or a management host with network access to a PVE 9.x cluster's API (tcp/8006) — the tool does not need to run on a cluster member.
- **A Prometheus** already receiving PVE's per-disk `blockstat` metrics via the InfluxDB output plugin and Telegraf. If your cluster's dashboards already show per-VM disk I/O, this is already true; `pve-storage-drs verify-metrics` (page 2) confirms it before you rely on it. No new exporter or collector is required — see `IMPLEMENTATION_PLAN.md` section 3.1 if you want the detail of where the data comes from.
- **A PVE user or API token** with read access to VM and storage inventory and permission to call `move_disk`. An API token is preferred for unattended operation (`execution.mode: auto`); a username/password pair works for interactive use. See "Setting up the PVE credential" below for the exact privileges — the obvious-looking "just grant Audit everywhere" is not enough, and the gap is silent rather than an error.

### Setting up the PVE credential

Grant, on **every storage the tool will manage** (the shared, image-holding storages in your `groups`, not backup targets):

- `Datastore.Audit` **and** **`Datastore.Allocate`** — not Audit alone. `GET /nodes/{node}/storage/{storage}/content` which the tool uses to cross-check each disk's real allocated size, silently returns an empty list under Audit-only permission: a clean `200` response with no content and no error, indistinguishable from "this storage genuinely has nothing on it" unless you already know to be suspicious. Audit alone is enough for every *other* read call the tool makes; this one endpoint is the exception, verified against a live PVE 9.2.11 cluster (see `IMPLEMENTATION_PLAN.md` section 3.5's note). Getting this wrong does not make the tool fail loudly — it makes it under-count what already occupies each storage, which quietly erodes the section 5.3 snapshot reserve it exists to protect.
- Grant it **at each storage's own path** (`/storage/<id>`), not only at the parent `/storage` — that is the grant confirmed to work.
- Also add `VM.Audit` cluster-wide (`/`) for VM inventory and config, which `PVEAuditor` already includes if you use that built-in role as a base.

**If you are using an API token** (recommended — see above), Proxmox's privilege separation means **the token has its own, separate ACL entries from its owning user**. A role granted to the user alone has no effect on the token, and a role granted to the token alone has no effect either unless the user has it too: the token's effective permission is the *intersection* of the two. Grant the same roles to both the user and `user@realm!tokenid`, or disable privilege separation on the token (Datacenter → Permissions → API Tokens → uncheck "Privilege Separation") so it always matches its user's permissions exactly.

A concrete, minimal setup: create a role (e.g. `pve-storage-drs`) with `Datastore.Allocate,Datastore.Audit,VM.Audit`, then add it as an ACL entry for both the user and the token at `/` for `VM.Audit`, and at each managed storage's `/storage/<id>` path for the datastore privileges.

**Two further, VM-level privileges are needed once `apply` executes a move** — `VM.Config.Disk` and `VM.Migrate`, both cluster-wide (`/`) or at least on every VM whose disks live in a managed group, granted to both the user and the token exactly as above. Neither is needed for anything this build actually runs today: `verify-metrics`, `show-load` and `verify-storages` only read, and only `move_disk` (called by `apply`, not yet implemented — `30-safety-and-status.md`) needs them. Grant them now alongside the privileges above so the credential does not need revisiting later; if you set up the credential read-only for now, add `VM.Config.Disk,VM.Migrate` to the role before you first run `apply`.

### Installing the package

On a Debian trixie host (which is what Proxmox VE 9.x is built on):

```
apt install pve-storage-drs
```

This installs the `pve-storage-drs` executable, its manpage, the example configuration at `/usr/share/doc/pve-storage-drs/examples/drs.example.yaml`, and this manual and the specification as PDFs under `/usr/share/doc/pve-storage-drs/`.

`coinor-cbc` and `python3-pulp` are `Recommends`, not `Depends`: without them the tool still plans, using the dependency-free heuristic of `IMPLEMENTATION_PLAN.md` section 5.5, but a Debian install should have them — `apt install pve-storage-drs` pulls them in by default.

### Where the configuration lives

`/etc/pve/drs.yaml`. On a cluster member this path is on `pmxcfs`, the cluster filesystem, so the file is automatically the same on every node — there is exactly one copy and no question of which one is authoritative. A management host that is not itself a cluster member simply keeps an ordinary file at that same path.

Two things follow from that placement, and both matter before you put a password in the file:

- **An edit is live on every node the instant it is saved.** There is no staging step. This is why configuration mistakes are always hard failures (see `30-safety-and-status.md`) rather than warnings, and why dry-run is the default execution mode.
- **Files under `/etc/pve` are group-readable by `www-data`** — the PVE web server's user. Check `ls -l /etc/pve/drs.yaml` on your cluster before deciding whether a plaintext `password:` is acceptable; the safer choice is an API token with `PVE_TOKEN_SECRET` set in the environment (or a systemd credential) rather than written into the file at all.

Copy `/usr/share/doc/pve-storage-drs/examples/drs.example.yaml` to `/etc/pve/drs.yaml` and edit it — every key is commented with its default and what it does. The full reference is `10-configuration.md`.

### Where the configuration is *not*

`state.path` (default `/var/lib/pve-storage-drs/state.json`) is deliberately **not** on `/etc/pve`. It is rewritten on every run, `pmxcfs` writes need cluster quorum, and the state it holds — cooldown timestamps, the load vector at the last balance, the run lock — is meaningful only to the one host that is actually running the timer. See `10-configuration.md#statepath`.

### Running the timer on exactly one host

Putting the configuration on `pmxcfs` does not make the tool cluster-aware. `state.json` is node-local: each node would keep its own cooldowns, its own drift baseline and its own run lock, which do not coordinate with each other. **Enable the systemd timer on exactly one host.** The one thing that *does* cross the cluster is a startup scan for in-flight `move_disk` UPIDs owned by the DRS user, which is what keeps two accidental instances from actively conflicting (they degrade to "slow and redundant" instead).

### First steps after installing

1. `pve-storage-drs -c /etc/pve/drs.yaml verify-metrics` — confirms the configured metric and label names actually exist in your Prometheus. See `20-verifying-metrics.md`.
2. Read `10-configuration.md` and adjust the storage groups, thresholds and weights for your cluster.
3. See `30-safety-and-status.md` for exactly which commands this build supports end to end today, and which still report "not implemented yet".

## Configuration reference

Every key pve-storage-drs reads is documented here: type, unit, default, what it interacts with, and what happens if you set it too high or too low. A key that exists in the schema but not here, or here but not in the schema, is a bug — tests/unit/test_documentation.py asserts the two never drift apart.

config/drs.example.yaml in the source tree (installed to /usr/share/doc/pve-storage-drs/examples/drs.example.yaml) carries the same defaults with inline comments; this page is the full explanation each comment points back to.

**schema_version**

Integer, required, no default.

The config format's version. pve-storage-drs refuses to start on a schema_version whose **major** number it does not understand, naming the version it does understand in the error, rather than guessing at fields from a future incompatible format. Currently 1.

### proxmox — the cluster API connection

**proxmox.host**

String, required, no default.

Any cluster member's hostname or IP. The API is reached at https://<host>:<port>/api2/json.

**proxmox.port**

Integer, default 8006.

The PVE API port. Change this only if you have put something unusual (a reverse proxy) in front of the API.

**proxmox.verify_ssl**

Boolean, default true.

Whether to verify the API's TLS certificate. Set false only for a self-signed certificate you cannot otherwise trust, and prefer proxmox.ca_file instead where possible — disabling verification also disables protection against a compromised network path to every node's API.

**proxmox.ca_file**

String path or null, default null.

A CA bundle to verify the API's certificate against, when it is not signed by a certificate already trusted by the host running pve-storage-drs.

**proxmox.auth.username**

String or null.

A PVE user, e.g. drs@pve. Required unless proxmox.auth.token_id is set. Username/password authentication is what the original requirement asked for; an API token (below) is preferred for execution.mode: auto because it needs no interactive ticket refresh and can be scoped to exactly the privileges section 3.5 of the plan lists.

**proxmox.auth.password**

String or null, default null, environment: PVE_PASSWORD.

Leave this null in the file and set PVE_PASSWORD in the environment (or a systemd credential) instead, especially when the config lives on /etc/pve and is therefore group-readable by www-data. A password given in the file is used as-is and is **not** overridden by the environment variable — see 00-installation.md for why the file's own permissions matter here.

**proxmox.auth.token_id**

String or null, default null.

An API token id, e.g. drs@pve!balancer. Set this instead of username/password for unattended operation; a token needs no ticket refresh loop and can be revoked independently of the user's password.

**proxmox.auth.token_secret**

String or null, default null, environment: PVE_TOKEN_SECRET.

The secret half of the API token, with the same environment-variable preference as proxmox.auth.password.

**proxmox.ticket_refresh_seconds**

Duration, default 3000 (50 minutes).

How often a username/password ticket is refreshed. PVE tickets last approximately two hours; refreshing well before that avoids a mid-run authentication failure. Not used for API-token authentication, which needs no ticket. Implemented by overriding proxmoxer's own (otherwise fixed-at-3600s) refresh interval after login, since proxmoxer has no constructor option for it; if a ticket somehow still gets rejected mid-run regardless (a long confirm-mode wait, a suspended process, a clock jump — see 50-pve-api.md), the client transparently logs in again from scratch and retries the one call that failed, once, before giving up.

**proxmox.read_workers**

Integer, default 12.

Size of the bounded thread pool used to fetch per-VM configuration — GET /nodes/{node}/qemu/{vmid}/config has no batch form, so this is what keeps a several-hundred-VM cluster's topology read from being serial. Raising it trades API server load for wall-clock time; 8–16 is the range the plan suggests. Too high a value on a small PVE API server can itself become the bottleneck. Only the per-VM fetch is parallelized; the cluster-wide and per-storage calls (50-pve-api.md) still run once each, sequentially, before it starts.

### prometheus — the metrics source

**prometheus.url**

String, required, no default.

The base URL of your Prometheus, e.g. http://prometheus.example.com:9090.

**prometheus.timeout_seconds**

Duration, default 30.

Per-request timeout against Prometheus. Too low a value on a busy or long-range query makes verify-metrics and the load model fail spuriously; too high a value delays noticing that Prometheus is actually unreachable.

**prometheus.username / prometheus.password**

String or null, default null.

HTTP basic auth credentials, if your Prometheus requires them. Ignored when prometheus.bearer_token is set.

**prometheus.bearer_token**

String or null, default null.

A bearer token for Prometheus instances behind an auth proxy that expects one. Takes precedence over basic auth when both are set.

### `metrics` — Telegraf/InfluxDB name mapping

Telegraf's naming is deployment-specific (`IMPLEMENTATION_PLAN.md` section 3.3), so every name below is configuration, never a constant. Run `pve-storage-drs verify-metrics` after changing any of them — see `20-verifying-metrics.md`.

**`metrics.read_ops`**

String, default `blockstat_rd_operations`.

The Prometheus metric name for per-disk read operations/second (raw counter, before `rate()`).

**`metrics.write_ops`**

String, default `blockstat_wr_operations`.

As `metrics.read_ops`, for writes.

**`metrics.read_bytes`**

String, default `blockstat_rd_bytes`.

The Prometheus metric name for per-disk bytes read (raw counter).

**`metrics.write_bytes`**

String, default `blockstat_wr_bytes`.

As `metrics.read_bytes`, for writes.

**`metrics.read_time_ns`**

String, default `blockstat_rd_total_time_ns`.

The Prometheus metric name for cumulative read I/O time in nanoseconds. This is the **primary** load signal by default (`load_weights.iotime`) — see `IMPLEMENTATION_PLAN.md` section 4 for why I/O time, rather than IOPS or bytes, is the right default.

**`metrics.write_time_ns`**

String, default `blockstat_wr_total_time_ns`.

As `metrics.read_time_ns`, for writes.

**`metrics.labels.vmid`**

String, default `vmid`.

The Prometheus label carrying the VM id.

**`metrics.labels.device`**

String, default `instance`.

The Prometheus label carrying the drive id (`scsi0`, `virtio0`, ...). **Confirm this with `verify-metrics`.** PVE's own tag of this name collides with Prometheus's own scrape-target `instance` label, and many Telegraf configurations rename or overwrite it; `verify-metrics` warns loudly when this is still set to the literal string `instance`.

**`metrics.labels.node`**

String, default `nodename`.

The Prometheus label carrying the PVE node name.

**metrics.rate_window**

Duration, default 5m.

The inner range passed to rate(). Must be at least four times metrics.pvestatd_push_interval, or rate() sees too few points to be meaningful — verify-metrics measures the *observed* sample spacing of a live series and errors if this rule is violated against reality, not just against the declared pvestatd_push_interval.

**metrics.step**

Duration, default 5m.

The sampling step used for range queries (forecasting, coverage checks). Independent of metrics.rate_window, though the two are usually set equal.

**metrics.pvestatd_push_interval**

Duration, default 60s.

How often PVE's pvestatd pushes metrics — a PVE-side setting this tool cannot read from the API, so it must be declared here. verify-metrics cross-checks it against the *observed* sample spacing of a live series and errors if they disagree by more than 20%, which is what stops a stale declaration from silently invalidating the rate_window rule above.

### window — the decision window

This is the period whose load is balanced. It is **not** the amount of history a forecaster needs to fit — see forecast.model below and IMPLEMENTATION_PLAN.md section 10.1.

**window.lookback**

Duration, default 24h.

The trailing window each disk's load is reduced over. Must be at least as long as the configured forecast.model needs (forecast.model: holt_winters at its default seasonal_periods needs 48h, which a 24h lookback can never supply — this is rejected at startup, not silently degraded).

**window.quantile**

Fraction in (0, 1), default 0.95.

The point-estimate quantile: each disk's load is the p95 of its raw signal over window.lookback, not the mean, so one traffic spike neither triggers nor suppresses a migration.

**window.upper_quantile**

Fraction in (0, 1), default 0.99.

The quantile the section 7.3 **saturation guard** actually consumes (when a storage configures saturation_load) — must be >= window.quantile. Being wrong in the direction of "busier than it looks" costs the guard deferring a move that was actually safe; the other direction risks the guard missing a mirror that pushes a storage past saturation. The optimizer itself still decides placement from window.quantile, the point estimate — see IMPLEMENTATION_PLAN.md §10.1's "As built" note; wiring the upper bound into the optimizer's own input remains future work.

**window.min_coverage**

Fraction in (0, 1], default 0.80.

A disk whose sample coverage over window.lookback falls below this fraction has its data rejected for this run; its last known load from state.json is used instead, flagged in the report. Never treated as zero — that would silently invite migrations *onto* a busy but under-sampled storage.

### `load_weights` — combining read/write and time/ops/bytes

See `IMPLEMENTATION_PLAN.md` section 4 for the full blend-and-rescale formula; the summary is that `iotime`/`ops`/`bytes` are blended in normalized space and the whole result is rescaled back onto the in-flight-I/O scale, so $\ell$`_d = raw_t(d)` exactly under the defaults below.

**`load_weights.iotime`**

Weight, default `1.0`.

Weight on the I/O-time term (`rate(rd_total_time_ns + wr_total_time_ns) / 1e9`), the primary signal by default: it is Little's-law in-flight I/O, additive across disks and self-weighting between large sequential and small random access with no manual read/write tuning needed.

**`load_weights.ops`**

Weight, default `0.0`.

Weight on the operations/second term. Non-zero only if you want IOPS to influence placement independently of I/O time.

**`load_weights.bytes`**

Weight, default `0.0`.

Weight on the bytes/second term, for the same reason as `load_weights.ops`.

**`load_weights.read_factor`**

Weight, default `1.0`.

Read/write asymmetry applied to the `ops`/`bytes` terms only (see `load_weights.write_factor`). Applying it to the `iotime` term as well double-counts, since I/O time already reflects the real read/write cost difference — do that only deliberately.

**`load_weights.write_factor`**

Weight, default `1.0`.

As `load_weights.read_factor`, for the write side. A write-heavy RAID-6 array, for example, might set this above 1.0 on the `ops`/`bytes` terms to express the real cost asymmetry those terms cannot otherwise see.

### `groups` — storage groups

**`groups[].name`**

String, required.

A group's name. A disk may only ever move between storages in its own group; there is no cross-group balancing (`IMPLEMENTATION_PLAN.md` section 5). Must be unique.

**`groups[].storages[].id`**

String, required.

A PVE storage id, or a `/regex/` pattern — a value that both begins and ends with `/` — matching one or more storage ids (`IMPLEMENTATION_PLAN.md` section 11.4). A pattern is matched with `re.fullmatch` (case-sensitive) against the live cluster's storage inventory on every run, so `/san-.*/` picks up a LUN added after the config was written with no edit needed; its own entry's `capability_weight`/`reserve_factor`/`saturation_load` apply to every storage it matches. A literal entry always overrides a pattern that also matches its storage, so one member of a pattern-matched family can still be pinned to different options. A storage may belong to **at most one** group (after pattern expansion) — belonging to two would make a disk's eligible destinations ambiguous, and config/cluster validation rejects it, as does a pattern matching zero storages or two patterns in one group claiming the same storage.

**groups[].storages[].capability_weight**

Weight, > 0, default 1.0.

The storage's **relative** share of the group's load — u_s = L_s / c_s is what actually gets equalized, so a storage with half the spindles of its peers can be given 0.5 and will correctly be assigned half the load. Only the *ratios* within a group matter; the absolute value is meaningless and it is not a capacity.

**groups[].storages[].reserve_factor**

Weight or null, default null (inherits snapshot_reserve.factor).

Per-storage override of the group-wide snapshot reserve factor, for a storage whose snapshot behaviour genuinely differs from its peers.

**groups[].storages[].saturation_load**

Positive number or null, default null.

The storage's approximate queue depth — the number of concurrent I/O requests it services before latency climbs super-linearly — in the same units as the load model (average in-flight I/O). Used only by the section 7.3 saturation guard on a migration's mirror. **Has no safe default**: an idle storage's observed load is not its capacity, so leaving this null (the default) simply disables that one advisory check for the storage; the hard bounds (migration.max_single_move_duration, the transient reserve invariant) always apply regardless. Obtain a real value from the array's documented queue depth, or by observing where latency actually starts climbing.

### snapshot_reserve — the free-space floor

**snapshot_reserve.factor**

Weight >= 0, default 2.0.

Keep this many times the largest disk on a storage free at all times, including *during* a migration, not merely before and after — PVE 9's volume-chain snapshots allocate a new full-size volume per snapshot, which is what this protects against. This is the constraint the tool never trades against balance (IMPLEMENTATION_PLAN.md section 5.3, (C5)).

**snapshot_reserve.min_free_bytes**

Size, default 0.

An absolute floor, applied as reserve = max(factor * largest_disk, min_free_bytes). Matters when a storage's largest disk is small: with factor: 2.0 and a 10 GiB largest disk, the snapshot term alone would reserve only 20 GiB on a 20 TiB LUN.

**snapshot_reserve.count_foreign_volumes**

Boolean, default true.

Count volumes DRS does not manage (templates, ISOs, backups, other groups' disks, orphans) against a storage's used capacity. Strongly recommended: turning this off understates real usage and silently erodes the reserve it is meant to guarantee.

### gates — deciding whether to act at all

**gates.drift_threshold**

Fraction, default 0.10.

Re-planning is skipped until the load vector has moved by this much in L1, relative to the vector recorded at the last executed balance. Lower reacts sooner and migrates more; below roughly 0.05 the tool will start chasing noise on a busy cluster. Interacts with gates.imbalance_threshold: drift decides *whether to look*, imbalance decides *whether to act*.

**gates.imbalance_threshold**

Fraction, default 0.20.

The minimum relative spread across a group's storages before a plan is actually built. A group under this threshold is left alone even if it has drifted. Also, unrelatedly, the section 10.2 backtest error ceiling a non-quantile forecast.model must stay within to drive the section 7.3 saturation guard — see forecast.model above.

**gates.cooldown_per_disk**

Duration, default 24h.

A disk that moved within this long is pinned in place for planning purposes. Prevents a disk from being shuffled back and forth across two still-noisy storages.

**gates.cooldown_per_storage**

Duration, default 1h.

A storage involved in a migration within this long accepts no new incoming moves. **Must exceed the implied wipe time of that storage's largest disk** (largest_disk / saferemove_throughput) or the next run will plan onto a storage that is still draining and stall — pve-storage-drs verify-storages computes this and warns when the configured cooldown is too short.

### migration — cost, bandwidth and the payback rule

**migration.bwlimit_bytes_per_sec**

Size/s, default 209715200 (200 MiB/s).

Passed to move_disk as bwlimit (converted to KiB/s at the API call site — the API's own unit, never bytes/s). Also the divisor in the section 7.1 mirror-duration estimate used by the payback test.

**migration.source_load_weight**

Weight, default 1.0.

Additional in-flight I/O charged to the **source** storage while a mirror is running — a mirror is one sequential reader, so 1.0 is the natural value in the same units as the load model.

**migration.target_load_weight**

Weight, default 1.0.

As migration.source_load_weight, for the target (one sequential writer).

**migration.payback_horizon**

Duration, default 7d.

The horizon H over which a plan's imbalance reduction is assumed to persist — benefit = ΔE * H. Setting this to 0 disables the payback test entirely (IMPLEMENTATION_PLAN.md section 11.1 rejects that at config-load time: payback_horizon > 0 is required).

**migration.payback_ratio**

Weight > 0, default 10.0.

λ: a plan is only accepted if its total benefit is at least this many times its total migration cost. This is the numeric form of "migrating a very large disk might generate more traffic than it saves."

**`migration.max_single_move_duration`**

Duration, default `6h`.

A hard per-move ceiling on `duration_mirror + duration_wipe`. A move predicted to take longer than this is rejected outright regardless of the aggregate payback test.

**`migration.account_saferemove_wipe`**

Boolean, default `true`.

Include the post-move `saferemove` zeroing pass in cost and duration estimates. PVE's LVM `saferemove` defaults to 10 MiB/s, so a 1.5 TiB disk's cleanup (~44h) can dwarf its ~2.2h mirror; disabling this only if you have verified `saferemove` is off on every storage in a group understates real migration cost badly if that assumption is wrong.

**`migration.wipe_load_weight`**

Weight, default `1.0`.

In-flight I/O charged to the source for the whole `saferemove` wipe duration — both in the cost model and in the section 7.3 saturation guard, where a draining move charges this to its source and nothing to its target. The zeroing pass is one sequential writer, so `1.0` is the natural value.

**`migration.saturation_ceiling`**

Fraction in (0, 1], default `0.85`.

The fraction of a storage's `saturation_load` (not of `capability_weight`) a move may drive it to. Inactive for any storage whose `saturation_load` is `null` — the default, since there is no safe way to infer it.

**`migration.assume_thick_provisioning`**

Boolean, default `true`.

Cost and reserve arithmetic use each disk's *provisioned* size rather than its currently allocated size. Set `false` only for genuinely thin-provisioned storage, and note that allocation can *grow* during a move even then.

### `objective` — the solver's trade-off weights

See `IMPLEMENTATION_PLAN.md` section 5.4 for the full objective and section 14.3 for a worked demonstration of `beta_move_count` and `kappa_vm_affinity` choosing between competing plans.

**`objective.spread_metric`**

`l1` or `minmax`, default `l1`.

How imbalance is measured: `l1` (sum of each storage's deviation from the group's target utilization) or `minmax` (only the single hottest storage). `l1` is the default because `minmax` is indifferent to a second nearly-as-bad storage once the worst one is fixed.

**`objective.alpha_spread`**

Weight, default `1.0`.

Weight on the imbalance term. This and the other three weights below share one scale — see the worked arithmetic in `IMPLEMENTATION_PLAN.md` section 14.3 for what "a 0.25-request reduction" actually costs against one move.

**`objective.beta_move_count`**

Weight, default `0.25`.

Penalty per migration — the tunable implementation of "minimize the number of migrations" (`IMPLEMENTATION_PLAN.md`'s "Requirement interpretation" note: a strict minimum would refuse a second

cheap move that halves the remaining imbalance, which is not desired). Raising it trades balance quality for fewer, larger moves.

**`objective.gamma_move_bytes_per_tib`**

Weight per TiB, default `0.05`.

Penalty per TiB actually migrated, biasing against moving large disks specifically (as opposed to `beta_move_count`, which only counts moves).

**`objective.kappa_vm_affinity`**

Weight, default `0.50`.

Penalty per extra storage a VM's disks are spread across, counted only **within** a group (a VM split across two groups is structural and cannot be repaired by any migration, so it is not counted). A soft preference: a strong imbalance or a capacity constraint can legitimately override it.

**`objective.affinity_counts_pinned_disks`**

Boolean, default `false`.

Whether a disk that cannot move this run (snapshot-blocked, excluded, or on a locked VM) still counts toward `kappa_vm_affinity`. Default `false` so one unreachable disk cannot veto good placement of the rest of its VM's disks; note `efidisk0`/`tpmstate0` are *not* in this pinned set on PVE 9.2 — they move online.

**`objective.reserve_violation_penalty`**

Weight, default `1000.0`.

A **floor**, not the value actually used, for the single-stage big-M fallback solve path (`IMPLEMENTATION_PLAN.md` section 5.3, option 2): the engine computes a provably-dominant `P` from the group's own load and disk sizes at solve time and uses `max(configured, computed)`, warning when it had to raise it. The default lexicographic two-stage solve (option 1, and the default) needs no penalty at all and is unaffected by this key.

### `solver` — which backend plans

**`solver.backend`**

One of `auto`, `cpsat`, `cbc`, `heuristic`; default `auto`.

`auto` prefers CP-SAT (`pip install proxmox-storage-drs[solver]` – not packaged for Debian), falls back to CBC (the packaged solver path via `python3-pulp` + `coinor-cbc`), then the dependency-free heuristic. `cpsat`/ `cbc` force one specific backend, failing that group's solve back to the heuristic (never a silent substitution of the *other* MILP backend) if its library is not importable or it cannot solve within `solver.time_limit_seconds` – force a specific backend only to reproduce or compare a result. `heuristic` skips the solver entirely. See `docs/internals/91-optimize.md`.

**`solver.time_limit_seconds`**

Duration, default `60`.

Wall-clock budget for a MILP solve attempt before falling back to the heuristic. Never emits a partial/unvalidated assignment on timeout — it falls back cleanly instead.

**`solver.mip_gap`**

Fraction, default `0.02`.

Acceptable optimality gap for the MILP solve. `IMPLEMENTATION_PLAN.md` section 5.5's coefficient-scaling error bound is three orders of magnitude below this, so it cannot itself change which plan is selected.

**solver.heuristic_iterations**

Integer > 0, default 5000.

Iteration budget for the heuristic's local-search descent phase (section 5.5). Higher can find a better local optimum on a large group at the cost of run time.

### execution — how (and whether) moves actually happen

**execution.mode**

One of dry-run, confirm, auto; default dry-run.

dry-run (the default) prints the plan and changes nothing. confirm executes one move at a time with a prompt before each. auto executes unattended, subject to the concurrency and time-window settings below. --mode on the command line overrides this for one run; moving *toward* less safety is logged at warning level (IMPLEMENTATION_PLAN.md section 11.3).

**execution.max_concurrent_migrations**

Integer >= 1, default 1.

How many moves may be in flight across the whole run at once, in --mode auto only (dry-run/confirm always run strictly sequentially regardless of this setting). Above 1 requires the *generalized* transient reserve invariant (section 8.1): several disks can land on one storage at once, and none of their sources release space until each individually completes — apply re-checks it live before launching each move. Launch order stays strictly FIFO: apply never reorders the scheduler's own queue to keep every slot busy, so a move that cannot launch yet is waited for rather than skipped past — see docs/manual/28-apply.md's own "Concurrent execution" section.

**execution.max_migrations_per_run**

Integer >= 1, default 5.

A ceiling on how many moves one invocation executes, independent of how many the plan contains — the remainder waits for the next run. auto mode only (dry-run/confirm execute or report the whole plan regardless); a shared budget across every group the run visits, not reset per group.

**execution.max_concurrent_per_storage**

Integer >= 1, default 1.

Caps concurrent moves touching one storage, counting it as either source or target, in --mode auto only. At the default of 1, two moves targeting the same storage serialize automatically and the generalized transient invariant collapses to its simple single-move form. Raising it should be a deliberate act on a storage with real spare headroom — two moves off the same source also means two concurrent saferemove wipes sharing one throttle.

**execution.max_replans_per_run**

Integer >= 0, default 3.

A cap on how many times one run may abandon its current plan and re-plan from newly observed state (section 9.2) — a mismatch between the plan and reality (a VM live-migrated mid-plan, a lock appeared) is normal, but a cluster churning faster than the engine can plan is a condition for a human, not for indefinite retrying. auto mode only — confirm/dry-run report a mismatch (replan_needed) and stop that group's own run for the operator to re-run by hand, rather than re-planning automatically.

**execution.abort_on_failure**

Boolean, default true.

On a failed move, stop the run and report rather than continuing with the remaining plan. The cluster is left in a valid intermediate state either way; the next run re-plans from reality.

**execution.poll_interval_seconds**

Duration, default 10.

How often an in-flight move_disk task's status is polled.

**execution.locks.wait_timeout**

Duration, default 4h.

How long to wait for a VM's config lock (any non-empty value — the set is treated as open-ended and never whitelisted) to clear before applying execution.locks.on_timeout. Generous by default: a backup of a large VM can easily run for hours, and that is a normal, not exceptional, condition.

**execution.locks.poll_interval**

Duration, default 30s.

How often the lock is re-checked while waiting.

**execution.locks.on_timeout**

One of skip, abort; default skip.

What happens when execution.locks.wait_timeout elapses: skip this move and continue with the rest of the plan, or abort the run. Never "force" — that option does not exist, deliberately.

**execution.source_release.wait**

Boolean, default true.

Wait for the source volume of a completed move to actually disappear (and the VM to be unlocked) before considering the move done, rather than trusting the move_disk task's own OK status — with saferemove enabled, OK does not mean the source space is back yet. Set false only on storages verified not to wipe.

**execution.source_release.timeout**

Duration, default 48h.

Bound on the wait above, sized for a multi-TiB disk wiping at the default 10 MiB/s. Must be at least largest_disk / saferemove_throughput for every storage where saferemove is on — verify-storages checks this.

**execution.time_windows[].days**

List of mon..sun, default: every day.

Which days a time window applies on. Applies only to execution.mode: auto — dry-run and confirm are not time-restricted, since neither changes anything without a human already present.

**execution.time_windows[].start / execution.time_windows[].end**

HH:MM strings, required together, must differ.

The window auto mode is permitted to execute moves in, in the **local time of the host running pve-storage-drs** — the same convention systemd.timer's own OnCalendar= uses by default — not UTC. A window may cross midnight (start: "22:00", end: "06:00"); start == end is rejected as ambiguous — it would silently mean either "never" or "always". No time_windows configured at all means no restriction: auto may execute at any time (section 2.1's "the engine may plan at any time and simply decline to act outside the window" only applies once at least one window is configured). Before starting a move, auto also refuses it (and stops the group's run cleanly, without aborting a move already in progress) if its estimated duration would not finish before the window closes.

### `exclude` — what DRS never touches

**`exclude.vmids`**

List of integers, default `[]`.

VM ids whose disks are never moved.

**`exclude.disks`**

List of `"vmid:device"` strings, default `[]`, e.g. `["101:scsi1"]`.

Individual disks excluded regardless of their VM.

**`exclude.storages`**

List of storage ids, default `[]`.

Storages excluded as both source and target — effectively removes them from their group without editing `groups`.

**`exclude.tags`**

List of PVE tags, default `["no-drs"]`.

A VM carrying any of these tags opts out entirely.

**`exclude.skip_vms_with_snapshots`**

Boolean, default `true`.

Pre-filter disks with a snapshot or an unreferenced companion volume before they reach the API. Setting this `false` does **not** make such a move work — PVE rejects `move_disk delete=1` on a snapshotted volume regardless; it only removes the early, informative warning. Keep this `true`.

**`exclude.running_only`**

Boolean, default `true`.

Only running VMs generate load to balance; a stopped VM's disks are moved only if a capacity constraint requires it. Setting this `false` widens the movable set to stopped VMs' disks for capacity purposes but they still contribute zero load.

**`exclude.include_unused_disks`**

Boolean, default `true`.

Whether detached (`unusedN`) volumes are movable. They carry zero load, so the solver only relocates one to repair a capacity violation — never for balance, since there is no benefit to buy. Set `false` to pin them in place entirely; they still count via `snapshot_reserve.count_foreign_volumes` either way.

**`report`**

**`report.warn_pinned_load_fraction`**

Fraction in (0, 1], default `0.25`.

If pinned load (snapshots, locks, exclusions) exceeds this fraction of a group's total, the residual imbalance may be structural rather than a planning failure, and the report says so alongside the best achievable spread given the pins.

**`state`**

**`state.path`**

File path, default `/var/lib/pve-storage-drs/state.json`.

Local disk, one copy per host, deliberately never /etc/pve — see 00-installation.md. Losing this file is safe but not free: cooldowns and the drift baseline reset, so the next run may migrate sooner than intended. Treat it as state to back up, not as a cache.

show-load and plan both read this file (if present) for the drift gate's history; neither writes it, and a missing or unreadable file just means every group evaluates as if it had never been balanced before — see ../internals/15-state.md.

### forecast — history beyond the plain quantile

See IMPLEMENTATION_PLAN.md section 10 and proxmox_storage_drs/forecast.py.

**forecast.model**

One of quantile, seasonal_naive, holt_winters; default quantile.

quantile needs only window.lookback and no model fitting. seasonal_naive and holt_winters need more history than the decision window alone — see below — and pve-storage-drs refuses to start if window.lookback (or your Prometheus retention) cannot supply it, rather than silently falling back.

**Backtest-validated before use.** Only when the section 7.3 saturation guard is actually active (some groups[].storages[].saturation_load is set): a seasonal_naive/holt_winters model is fit on the older half of its own recent history and checked against what actually happened in the newer half, once per group, before it is trusted for that run. A model that misses by more than gates.imbalance_threshold — or that does not yet have enough history to backtest at all — falls back to quantile for that group's saturation guard this run, logged at warning (IMPLEMENTATION_PLAN.md section 10.2). quantile itself is never backtested; there is nothing to validate and nothing more conservative to fall back to.

**forecast.seasonal_lookback_days**

Days, default 7.

History seasonal_naive needs: same-hour-of-day samples across this many days. 0 does not disable the model or fall back to anything — it simply stops requiring history beyond window.lookback, which for a lookback under 24h can leave no same-hour-of-day sample to match at all (predicting 0.0, not a fallback).

**forecast.holt_winters.seasonal_periods**

Integer > 0, default 288 (24h at a 5-minute step).

Samples per seasonal cycle. Combined with metrics.step, this determines holt_winters's required history: 2 * seasonal_periods * metrics.step — 48h at the defaults, which a 24h window.lookback can never satisfy.

**forecast.holt_winters.trend**

One of add, mul, none; default add.

The trend component passed to the underlying Holt-Winters fit (statsmodels, an optional dependency — see 30-safety-and-status.md).

**forecast.holt_winters.seasonal**

One of add, mul, none; default add.

The seasonal component passed to the same fit.

**forecast.holt_winters.residual_z**

Weight, default 2.0.

The upper bound is point_estimate + residual_z * stdev(residuals) from the in-sample fit — this is what the section 7.3 saturation guard actually consumes (see window.upper_quantile for the equivalent under the quantile model; the optimizer itself does not consume this).

## Verifying your setup

Run this before relying on any plan. Telegraf and InfluxDB naming varies by deployment, so `pve-storage-drs` never assumes the defaults in `10-configuration.md` are correct for your cluster — `verify-metrics` is what checks them against the live Prometheus instead.

### Running it

```
pve-storage-drs -c /etc/pve/drs.yaml verify-metrics
```

A successful run looks like this (labels and values are illustrative):

```
[   info] metrics.read_ops = 'blockstat_rd_operations' exists
[   info] metrics.write_ops = 'blockstat_wr_operations' exists
[   info] metrics.read_bytes = 'blockstat_rd_bytes' exists
[   info] metrics.write_bytes = 'blockstat_wr_bytes' exists
[   info] metrics.read_time_ns = 'blockstat_rd_total_time_ns' exists
[   info] metrics.write_time_ns = 'blockstat_wr_total_time_ns' exists
[   info] blockstat_rd_operations: sample series labels {'vmid': '101', 'instance': 'scsi0', 'nodename':
'pve01'}
...

OK: verify-metrics found no blocking problems
```

Exit status is `0` when every check passes, `1` when any check reports an `error`-level finding (see below). `--json` emits the same information as one JSON object instead — see `30-safety-and-status.md` for exit codes in general.

### What each finding means

`verify-metrics` runs six checks, in this order (`IMPLEMENTATION_PLAN.md` section 3.3):

1. **Metric names exist.** Each of `metrics.read_ops`, `write_ops`, `read_bytes`, `write_bytes`, `read_time_ns` and `write_time_ns` is looked up against Prometheus's own `/api/v1/label/__name__/values`. A missing name is an **error** — fix the name in `10-configuration.md`'s `metrics.*` section to match what your Telegraf actually emits.
2. **Sample series and labels.** One instant query per metric, printing its full label set so you can see at a glance whether `metrics.labels.vmid`, `.device` and `.node` are really present and non-empty on a live series. A metric that exists but currently has no series is a **warning**, not an error — it may simply mean nothing has generated that kind of I/O recently.
3. **Configured labels present.** If `metrics.labels.vmid`/`.device`/`.node` do not appear (or are empty) on the sample series, that is an **error**: the load model has nothing to join disks on.
4. **The `instance` collision.** If `metrics.labels.device` is still the literal string `instance`, a **warning** — PVE's own per-disk tag has this name and it collides with Prometheus's unrelated scrape-target `instance` label. Many Telegraf configurations rename or overwrite one of the two; confirm which one survived before trusting this.
5. **Per-disk coverage.** For every `(vmid, device)` pair seen on `metrics.read_ops`, the fraction of expected samples actually present over `window.lookback` is measured. Below `window.min_coverage`, a **warning** naming the disk — that disk's own data will be rejected by the load model at plan time and its last known load from `state.json` used instead.
6. **Observed sample spacing.** The modal gap between consecutive samples of a live series is measured directly, rather than trusted from `metrics.pvestatd_push_interval`. Disagreeing by more than 20%, or `metrics.rate_window` being below four times what was actually observed, is an **error** — this is what stops a stale `pvestatd_push_interval` declaration from silently breaking the `rate()` window it is supposed to protect.

### If it fails

- **A metric does not exist**: check your Telegraf `influxdb_listener` (or equivalent) configuration for the actual measurement/field names it is producing, and update `metrics.*` to match.
- **A label is missing or empty**: the sample series printed by check 2 is the fastest way to see what labels Prometheus actually has for that series — update `metrics.labels.*` to name the real ones.
- **Coverage is low for specific disks**: check that Telegraf is actually scraping every node, and that `window.lookback` is not longer than your Prometheus retention.
- **Spacing disagrees**: measure `pvestatd`'s real push interval on a node (its systemd timer, or the interval visible in `/etc/pve/status.cfg` if set there) and correct `metrics.pvestatd_push_interval`.

## Reading `show-load` and `verify-storages`

Both commands only read — neither ever changes anything, in any `execution.mode`. Run them any time you want to see what the tool sees.

#### `show-load`

Prints every storage in every configured group, its disks, and its snapshot-reserve status. This example uses the group from `IMPLEMENTATION_PLAN.md` section 14's worked example (`config/drs.example.yaml` ships the same group and weights) so the numbers are traceable to that section rather than invented for this page:

```
$ pve-storage-drs -c /etc/pve/drs.yaml show-load
Group fc-tier1 → ACT: reserve violated on san-a; bypassing the drift and imbalance gates (section 13: safety
is not subject to hysteresis)
  san-a  used 4.50 TiB/8.00 TiB  L=6.50 u=6.50  ⚠ reserve short by 512.00 GiB  (largest disk 2.00 TiB,
  requires 4.00 TiB free)
    101:scsi0        2.00 TiB  raw     ℓ 3.00
    101:scsi1        1.00 TiB  raw     ℓ 1.00
    102:scsi0        1.50 TiB  raw     ℓ 2.50
  san-b  used 1.50 TiB/8.00 TiB  L=0.70 u=0.70  reserve OK  (largest disk 1.00 TiB, requires 2.00 TiB free)
    103:scsi0      512.00 GiB  raw     ℓ 0.40
    104:scsi0        1.00 TiB  raw     ℓ 0.30
  san-c  used 512.00 GiB/8.00 TiB  L=0.20 u=0.20  reserve OK  (largest disk 512.00 GiB, requires 1.00 TiB
  free)
    105:scsi0      512.00 GiB  raw     ℓ 0.20
```

`L=`/`u=` on a storage's line are section 4's `L_s` (summed ℓ of the disks currently on it) and `u_s = L_s / capability_weight`; ℓ after a disk's size/format is that disk's own share. Both come from `IMPLEMENTATION_PLAN.md` section 14's worked example, so the numbers above are traceable to that section rather than invented for this page.

#### The `Group <name> → ACT/NO ACTION` line

Section 6's three gates, evaluated in order — reserve override, then drift, then imbalance — and the first one that decides wins. The header line above shows the **reserve override**: san-a's 0.5 TiB shortfall forces `ACT` outright, regardless of how balanced or drifted the group is, because "safety is not subject to hysteresis" (section 13). The other two shapes this line can take, on a group with no reserve violation:

```
Group fc-tier1 → ACT: imbalance 255.4% meets or exceeds gates.imbalance_threshold (20.0%)
Group fc-tier1 → NO ACTION: imbalance 12.2% is below gates.imbalance_threshold (20.0%)
```

`show-load` reads `state.path`'s recorded load vector (section 11.2) and passes it to the drift gate, so once a group has one on record, this line can also read:

```
Group fc-tier1 → NO ACTION: drift 3.1% is below gates.drift_threshold (10.0%)
```

A group with no `state.json`, or none recorded for it yet, still evaluates as if this were the very first run — the drift gate is skipped outright (section 6's own degenerate-case rule), not "treated as zero drift" — so its verdict falls straight through to a reserve override or an imbalance check, exactly as before this was wired up. `docs/internals/80-gates.md` and `docs/internals/15-state.md` have the detail; nothing yet *writes* `last_balance` (that needs `execute.py`, not yet written), so this stays the common case until a migration has actually run.

A pinned disk carries `[pinned: <reason>]` after its size and format — `snapshots present (N)`, `locked: <lock>`, `excluded by config`, `excluded: unused disk (exclude.include_unused_disks=false)`, or `cooldown: moved recently, <time> left on gates.cooldown_per_disk` whenever `state.json` records that disk having moved within `gates.cooldown_per_disk` — matching `IMPLEMENTATION_PLAN.md` section 5.3 (C2) exactly; that reason is the answer to "why won't it move this disk."

A disk whose measured I/O falls below `window.min_coverage` (section 3.4) gets a `⚠ <disk> : <reason>` line of its own, right after that group's disks, instead of a silently wrong $\ell$ — it either shows a `state.json`-recorded last known load (see `docs/internals/70-loadmodel.md`) or `0.0` when no `state.json` entry exists for it, and either way the warning says which. `tpmstate0` and `unusedN` disks are never flagged this way: they genuinely emit no I/O metrics, so $\ell$ `0.00` for them is correct, not a gap. A group with no measured I/O at all gets one `(idle: no measured I/O for this group this window)` line instead of per-disk warnings.

**A Prometheus outage does not fail this command.** Sizes and reserve status never depend on Prometheus at all; if the load fetch for a group fails, that group's storage/disk lines simply omit `L=`/`u=`/$\ell$, and a `⚠ per-disk load unavailable: <reason>` line explains why — check `verify-metrics` first if you see this.

A **`Warnings:`** block, when present, lists things worth a look but not fatal: a disk on a storage that is not in any configured group ("ungrouped, not managed" — check whether that storage belongs in a group), or a disk whose size came from its own config rather than the storage's authoritative content listing. The latter usually means the volume no longer exists on the storage — most often because it was removed directly on the storage backend rather than through Proxmox, which leaves a dangling reference behind that PVE does not clean up or even notice on its own (a VM starts fine with one; see `IMPLEMENTATION_PLAN.md` section 3.6 for a confirmed case). If you see this warning, check whether the named volume still exists on the storage and, if it genuinely does not, remove the stale reference (`qm unlink <vmid> <device>` for an `unusedN` entry).

`--json` emits the same information as one object with `groups[].storages[]` (including the exact byte counts behind the reserve check, plus `load` and `utilization` when available) and `groups[].disks[]` (plus `load` and `load_flagged_reason` when available), with each group carrying its own `load_computed`, `idle`, `load_error` and `gate` fields — a Prometheus outage on one group never prevents another group's `load_computed: true` or `gate` from being reported. `gate` is `null` when `load_computed` is `false` (no `GroupLoad` to evaluate gates against) and otherwise an object with `act` (bool), `reason` (string, identical to the human line's text after the arrow), `reserve_override` (bool), and `drift_fraction`/ `imbalance_fraction` (float or `null` — `null` means that gate was never reached, not that it evaluated to zero).

**A config with several groups issues Prometheus queries per group.** Computing one group's load takes seven queries (six raw metrics plus one coverage check), unfiltered by group — the fastest correct way to get one group's numbers, but it means a config with `N` groups makes `7N` queries in total on every `show-load`, not 7, since each group's coverage and data-quality decisions are independent and so cannot share a single fetch. For the typical one-to-three-group deployment this is a handful of fast instant queries and not worth worrying about; an operator running many groups against an already-busy Prometheus should

be aware of the multiplier.

**verify-storages**

Reports saferemove and the wipe time it implies for the largest disk on each storage, and warns when your configured cooldown or move-duration limits are shorter than that implied wipe — the condition IMPLEMENTATION_PLAN.md section 9.3 describes as "the next run plans onto a storage that is still draining". Section 14's fixture has saferemove off everywhere (stated explicitly there, so its payback arithmetic is reproducible); this example instead assumes an LVM san-b with saferemove on at PVE's default 10 MiB/s throughput, to show what the warning looks like:

```
$ pve-storage-drs -c /etc/pve/drs.yaml verify-storages
Group fc-tier1
  san-a  saferemove=off
    saferemove is off or throughput unknown; no wipe-time check
  san-b  saferemove=on
    implied wipe time for the largest disk (1.00 TiB): 1.2d
    ⚠ gates.cooldown_per_storage (1.0h) is shorter than the implied wipe time -- the next run may plan onto
    a still-draining storage (section 9.3)
    ⚠ migration.max_single_move_duration (6.0h) is shorter than the implied wipe time -- a move of the
    largest disk would be rejected outright (section 7.3)
  san-c  saferemove=off
    saferemove is off or throughput unknown; no wipe-time check
```

Storage types without saferemove at all (Ceph RBD, ZFS) always report saferemove=off and skip the check — there is nothing to wipe.

If any groups[].storages[].id is written as a /regex/ pattern (IMPLEMENTATION_PLAN.md section 11.4), verify-storages also prints what each one matched this run, and lists any cluster storage matched by no group at all — the same "ungrouped, not managed" visibility show-load gives per disk, but at the storage level and independent of whether a disk currently happens to be on it:

```
Pattern expansions (section 11.4):
  [fc-tier1] /san-.*/ → san-a, san-b, san-c

Cluster storages matched by no group:
  - local-only
```

An over-broad or dead pattern is the one new way to misconfigure a group this feature adds, so this expansion is never silent: every run also logs it at INFO, whether or not verify-storages is the command being run.

## Reading plan

plan computes what pve-storage-drs would do and prints it. It never changes anything, in any execution.mode — that is apply's job (see docs/manual/28-apply.md), and it needs a separate, explicit invocation to run at all.

For each group, plan:

1. Fetches the same load (ℓ_d/L_s/u_s) show-load does, and evaluates the same section 6 gate.
2. If the gate says NO ACTION, stops there for that group — no solver runs, nothing more to show.
3. If it says ACT, solves it with whichever backend solver.backend selects (IMPLEMENTATION_PLAN.md section 5.4/5.5 — see docs/internals/91-optimize.md for the CP-SAT/CBC backends and docs/internals/90-heuristic.md for the dependency-free one) to compute a target assignment, orders its moves under the section 8 transient reserve invariant, and checks the whole plan against section 7's payback rule.

This example uses the same section 14 worked example `show-load`'s manual page does, at the default weights (the three-move plan) and `solver.backend: auto` with neither MILP library installed, so it falls back to the heuristic — the numbers are traceable to that section either way, since every backend reports through the identical objective function:

```
$ pve-storage-drs -c /etc/pve/drs.yaml plan
Group fc-tier1 → ACT: reserve violated on san-a; bypassing the drift and imbalance gates (section 13: safety
is not subject to hysteresis)
  solver: heuristic
  1. 102:scsi0      san-a → san-c     1.50 TiB   ~2.2h   Δimbalance -4.53   ℓ/z 1.67
  2. 101:scsi1      san-a → san-b     1.00 TiB   ~1.5h   Δimbalance -2.00   ℓ/z 1.00
  3. 105:scsi0      san-c → san-b   512.00 GiB   ~43.7m   Δimbalance -0.40   ℓ/z 0.40
  after: san-a=3.00  san-b=1.90  san-c=2.50
  spread: 255.4% → 44.6%
  payback: benefit 4.19e+06 load·s vs cost 3.15e+04 load·s → ratio 133 (need 10) ✓
```

The `solver:` line names whichever backend actually produced this plan – `heuristic`, `cpsat` or `cbc` – plus, for a MILP backend, `(optimal)` or `(feasible)` (the gap was not proven closed within `solver.time_limit_seconds`, but a solution was still found). It is not always what `solver.backend` says: `auto` tries CP-SAT then CBC before the heuristic, and even an explicitly forced `cpsat`/`cbc` falls back to the heuristic (logged as a warning in that case) rather than failing the whole run, if that library is not installed or cannot solve within the time limit – see `docs/internals/91-optimize.md`.

The `after:`/`spread:` lines (and the payback numbers) reflect what the scheduler actually managed to order, not the solver's target assignment — identical here since all three moves scheduled cleanly, but see the deadlock note below for when a plan cannot schedule everything it proposed.

A group the gate does not act on prints one line and stops:

```
Group fc-tier2 → NO ACTION: imbalance 0.0% is below gates.imbalance_threshold (20.0%)
```

### Reading a move line

`1. 102:scsi0 san-a → san-c 1.50 TiB ~2.2h Δimbalance -4.53 ℓ/z 1.67` — the disk, its current and target storage, its size, the estimated mirror duration (`size / migration.bwlimit_bytes_per_sec`, plus a `+wipe <time>` suffix when `saferemove` on the source storage adds one — see the payback section below), this move's own effect on the group's imbalance metric at the moment it was scheduled (negative means imbalance went down, which is the usual case; a move scheduled mainly to resolve a reserve violation or consolidate a VM can show a positive value and still be correct), and `ℓ/z` — load per TiB, section 7.3's own "single best indicator of a good migration candidate: high I/O concentrated in a small disk."

A move whose own duration exceeds `migration.max_single_move_duration` carries a `⚠ exceeds migration.max_single_move_duration` suffix and always fails the plan (see below) — this is a hard, per-move rule, independent of whether the plan as a whole looks profitable.

`resolves_reserve_violation` (visible in `--json`, and implied by the `ACT: reserve violated on ...` header in human output) marks a move scheduled first *regardless* of its ratio, because the storage it leaves is currently breaching (C4)/(C5) — safety is not subject to hysteresis (section 13), so this always wins over a purely balance-driven move.

**A ⚠ line means a deadlock, not a hidden failure.** If the target assignment includes a move this run cannot find any transient-feasible order for, `plan` says so explicitly (`IMPLEMENTATION_PLAN.md` section 8.3's "report, never force" path — staging and plan-splitting are not yet implemented, so this is where a genuine cycle or an unavoidable capacity shortfall currently surfaces) rather than silently omitting the move or, worse, telling you a plan is clean when part of it is not achievable. When only *some* moves

deadlock, the `after:`/`spread:`/`payback:` lines report the state reachable by the moves that did schedule, never the fuller picture the undeliverable ones would have produced.

### The `payback:` line

Section 7.3's acceptance test: `benefit >= migration.payback_ratio * cost`, both sides in load-seconds. `benefit` is `(E_before - E_after) * migration.payback_horizon` (section 7.2's own *unweighted* `E`, not the solver's `alpha_spread`-scaled objective term — the two agree numerically at the default `alpha_spread: 1.0`, but the payback ratio must not depend on that tuning knob); `E_after` is evaluated against what `plan`'s own scheduler actually managed to order, not the solver's aspirational target, so a partially-deadlocked plan is scored on the moves it can really make, not ones it cannot. `cost` sums every *scheduled* move's `duration_mirror * (source_load_weight + target_load_weight) + duration_wipe * wipe_load_weight` (section 7.1). A ✓ plan passed; a ✗ one did not.

A ✗ plan can fail for two different reasons, reported as two different lines, because they call for different fixes:

- **Economic failure** — the balance benefit does not outweigh the migration cost. Adjust `objective`'s weights, `migration.payback_ratio`, or accept the plan is not worth running.
- **Hard-duration failure** — some move's own `duration_mirror + duration_wipe` exceeds `migration.max_single_move_duration`, regardless of whether the plan as a whole is profitable. Fix `saferemove` throughput, `migration.bwlimit_bytes_per_sec`, or the duration limit itself.

Either, both, or neither can apply to the same plan; `plan` names exactly which happened rather than one generic "does not pass the payback test" line for both.

**A plan resolving a reserve violation always passes this test**, regardless of the ratio shown — the example above happens to pass on merit (133 ≥ 10), but a plan whose *only* move fixes a (C4)/(C5) violation with zero balance benefit (a real, common case: relocating the sole loaded disk in a two-storage group changes which side carries it without reducing spread at all) is accepted too. Section 13's "the reserve is never traded against balance" applies here exactly as it does to the gates: an operator does not get to decline a capacity emergency fix because it scores poorly against `migration.payback_ratio`. The hard per-move duration rule is not exempted this way — it is an operational limit, not an economic one, and still blocks the plan.

**What a ✗ (or a rejected move) does *not* do today: automatically make the plan smaller and retry.** Section 7.3 describes re-solving with `beta` and `gamma` doubled, up to three times, converging on the highest-value subset of moves. That loop is not implemented — a failing plan is reported and left for you to review, not silently adjusted. See `docs/internals/96-payback.md`.

### What `plan` does not yet do

- **No automatic re-solve-and-shrink on a failing payback test** (see above) — reported, not fixed for you.
- **The section 7.3 saturation-ceiling defer check only covers the mirroring phase**, not a second, separate check for the *draining* phase a `saferemove` wipe holds a storage in afterward — that needs `schedule.py` to reason about which moves actually overlap in time, which it does not do. The check itself is otherwise active for any storage that configures `saturation_load` (still none in this project's own dogfooding cluster) — a deferred move is reported separately from a hard-duration-rejected one (`deferred_moves`, not `rejected_moves`) and excluded from `apply` the same way. See `docs/internals/96-payback.md`.
- **`plan` itself still only reads `state.json`, never writes it.** Its gate reads real `last_balance` history when a group has one recorded (see `docs/internals/15-state.md`), and a disk/storage cooldown pins or excludes exactly as described below – but `plan` never executes a migration, so it never

has anything of its own to record. apply is what writes last_balance and cooldowns now, after a run that actually executed at least one migration (docs/manual/28-apply.md); a group with no state.json yet, or one apply has never touched, still evaluates as a first run (drift gate skipped).

- **Cooldowns.** A disk moved within gates.cooldown_per_disk is pinned (show-load/plan both show it as [pinned: cooldown: ...]), and a storage that was a migration's *destination* within gates.cooldown_per_storage accepts no new incoming moves from the heuristic. See docs/internals/15-state.md, docs/internals/60-topology.md and docs/internals/90-heuristic.md.
- **No staging, no concurrent scheduling.** Documented as deliberate, not forgotten, in docs/internals/95-schedule.md.

--json emits groups[], each with gate (identical shape to show-load's), solver_backend/solver_status (null/null when the gate said NO ACTION; otherwise "heuristic"/null, or "cpsat"/"cbc" with "optimal"/"feasible" – the same information the human output's solver: line names), moves[] (disk_key, vmid, device, from_storage, to_storage, size_bytes, imbalance_reduction, resolves_reserve_violation, load_per_tib, duration_mirror_seconds, duration_wipe_seconds, cost_load_seconds, exceeds_max_duration), deadlocked (a list of disk keys) and deadlock_message (null if none), before_spread/after_spread (the section 6 spread fraction, before the plan and after every scheduled move), load_error (null unless Prometheus failed for this group), and payback — null when there is no GroupLoad or the gate said NO ACTION, otherwise an object with benefit_load_seconds, total_cost_load_seconds, ratio, aggregate_ok (the economic test alone, or true if exempted), rejected_moves (disk keys failing the hard duration rule), deferred_moves (disk keys deferred by the section 7.3 saturation guard — empty unless a storage in the group configures saturation_load) and accepted (aggregate_ok and neither list non-empty).

## Reading apply

apply runs the identical per-group pipeline plan prints — the same gate, the same solver, the same scheduler, the same payback test (plan's own manual page, docs/manual/27-plan.md, describes all four in detail) — and then, unless it is dry-run, actually issues the moves. execution.mode (or --mode) picks what happens next:

| Mode | What apply does |
|---|---|
| dry-run *(default)* | Prints the same report plan would, plus a would_move outcome per move. Issues zero API calls. |
| confirm | Executes one move at a time, prompting before each: [y]es/[n]o skip/[a]ll remaining/[q]uit. |
| auto | Executes unattended, subject to execution.time_windows, max_migrations_per_run, and a bounded automatic re-plan loop — see "Reading auto mode" below. Also the only mode that honours execution.max_concurrent_migrations/max_concurrent_per_storage above their default of 1, running several moves at once — see "Concurrent execution" below. |

A confirm run, using the same section 14 fixture plan's own manual page walks through, with the operator declining the first move and accepting the rest:

```
$ pve-storage-drs -c /etc/pve/drs.yaml --mode confirm apply
Group fc-tier1 → ACT: reserve violated on san-a; bypassing the drift and imbalance gates (section 13: safety
is not subject to hysteresis)
  solver: heuristic
  102:scsi0  san-a → san-c  1.50 TiB  [y]es/[n]o skip/[a]ll remaining/[q]uit? n
  1. 102:scsi0      san-a → san-c     1.50 TiB   ~2.2h   Δimbalance -4.53   ℓ/z 1.67  → skipped: operator
  declined
```

```
101:scsi1  san-a → san-b  1.00 TiB  [y]es/[n]o skip/[a]ll remaining/[q]uit? a
2. 101:scsi1      san-a → san-b     1.00 TiB   ~1.5h   Δimbalance -2.00   ℓ/z 1.00  → moved: task UPID:...
completed OK, source released
3. 105:scsi0      san-c → san-b   512.00 GiB   ~43.7m   Δimbalance -0.40   ℓ/z 0.40  → moved: task UPID:...
completed OK
after: san-a=3.00  san-b=1.90  san-c=2.50
spread: 255.4% → 44.6%
payback: benefit 4.19e+06 load·s vs cost 3.15e+04 load·s → ratio 133 (need 10) ✓
```

Everything above the `→ status: detail` suffix on each move line is identical to `plan`'s own output — the "after"/"spread"/"payback" lines still describe the plan as scheduled, not adjusted for what a declined move would have changed, since (as with a partial deadlock, see `plan`'s manual page) reporting what was actually asked for is more honest than recomputing a hypothetical.

### The `[y]es/[n]o skip/[a]ll remaining/[q]uit` prompt

Only in `confirm` mode, once per move, before it is issued — never for a move `dry-run` would only report. `[a]ll remaining` stops prompting for the rest of *this run* (every group, not just the current one); `[q]uit` stops the whole run immediately, including any group not yet even planned. An answer that is not one of the four is rejected right there ("please answer y, n, a or q") and asked again — it never reaches `execute.py` at all, so a typo can never be mistaken for one of the four real decisions.

### Reading `auto` mode

`auto` behaves like `[a]ll remaining` chosen up front, unprompted, plus three things `dry-run`/`confirm` do not do at all (`IMPLEMENTATION_PLAN.md` section 9.1/9.2):

- **`execution.time_windows`.** If any are configured, a move is refused (reported `skipped`, "outside execution.time_windows") unless the current moment — in the **local time of the host running `pve-storage-drs`**, not UTC — falls inside one of them, and before starting a move `auto` also refuses it if its estimated duration (mirror plus any `saferemove` wipe) would not finish before the window closes. No windows configured at all means no restriction. Either way, a move already in progress is never aborted at window close — the check only ever runs *before* issuing the next one.
- **`execution.max_migrations_per_run`.** A ceiling on how many moves the whole invocation executes, shared across every group it visits (not reset per group) — the remainder waits for the next scheduled run.
- **The re-plan loop.** A `replan_needed` mismatch does not end an `auto` run the way it ends a `dry-run`/`confirm` one: `apply` re-invokes the whole pipeline (gates, load model, solver, payback, ordering) from freshly observed cluster state and tries again, up to `execution.max_replans_per_run` times. The gate may well conclude no further action is needed on the re-plan — a normal, quiet outcome, not a failure. Exceeding the cap does end the run, with a message naming the setting: "a cluster churning faster than the engine can plan is a condition for a human to look at, not to iterate against." Cross-referencing the report's `outcomes[]` shows the whole story for a re-planned group: the mismatch that triggered each re-plan, and whatever the next attempt then did.

### Concurrent execution

Configuring `execution.max_concurrent_migrations`/`max_concurrent_per_storage` above their default of `1` runs several moves at once in `auto` mode (`dry-run`/`confirm` always run strictly sequentially, regardless of these settings — concurrency only makes sense for unattended operation). Section 8.1's transient reserve invariant generalizes to a whole in-flight set of moves landing on the same storage at once, checked live before every launch exactly like the sequential executor's own pre-flight re-check; `max_concurrent_per_storage` counts a storage as occupied whether a move touches it as source *or* target.

**Launch order stays strictly FIFO.** The scheduler's own queue (the same one a sequential run would follow, one move at a time) is never reordered to keep every concurrency slot busy: if the next queued move cannot launch yet — its VM is locked, or a per-storage cap is already reached — `apply` waits for it rather than skipping ahead to a later move that could launch immediately. Several moves genuinely run at once once launched (nothing blocks waiting for one move's own completion or lock to affect any other), but which move launches *next* is always decided in the order `plan` would have shown it. This can under-deliver on throughput in a mixed queue, but never launches a move out of the order the payback-scored plan itself computed.

A failure (or a budget/deadline limit) stops the run from launching anything further, exactly as in the sequential case, but does not abandon moves already in flight: `apply` keeps polling them to their own natural conclusion (`moved`, `failed`, or `draining`) before returning, so nothing already running is left unreported. Section 7.3's saturation guard is enforced at planning time under concurrency exactly as under the sequential executor (a flagged move never reaches either executor); what neither executor does is re-check the ceiling *during* execution against the live in-flight set (`docs/manual/30-safety-and-status.md`'s own note on that narrower gap applies here too).

### What "done" means for one move, and why it can take a while

A move is not finished when the `move_disk` task itself succeeds. `IMPLEMENTATION_PLAN.md` section 9.3 requires all three, together: the task reports `exitstatus: OK`, the source volume is gone from the source storage's own content listing, and the VM's config lock is clear again. A storage with `saferemove` enabled zeroes the old volume afterwards at `saferemove_throughput` (default 10 MiB/s) — for a large disk, that can run for hours after the task itself is long done. While it runs, `apply` reports the move as `draining`, not `moved` and not `failed`: it is a normal, expected state, bounded by `execution.source_release.timeout` (default 48h), not an error. A `draining` move still counts as "executed" for `state.json`'s bookkeeping below — the mirror itself completed; only the source's own cleanup is still in flight. For the *rest of this same run*, that storage is excluded as both a source and a target for any later move: it holds a storage-level lock for as long as the wipe runs, so a subsequent `move_disk` touching it would either queue behind a wipe that can take days or fail outright. A move skipped this way is reported plainly (`skipped`, naming the draining storage) rather than attempted into that lock.

Before *every* move, `apply` re-checks the live cluster rather than trusting the plan: the VM may have moved node, the disk may no longer be on the expected source, a snapshot may have appeared, it may have been tagged for exclusion, or the target storage's free space may no longer satisfy the section 8.1 transient invariant. Any of these stops the group's run with `replan_needed`. In `dry-run`/`confirm`, that is the end of it — the operator re-runs `apply` by hand once ready; see "Reading `auto` mode" below for what `auto` itself does about it automatically.

A VM config lock (`backup`, `snapshot`, `migrate`, or any other value — this set is never whitelisted, see `docs/internals/92-execute.md`) makes `apply` wait, not fail, up to `execution.locks.wait_timeout_seconds`. What happens on that timeout depends on `execution.locks.on_timeout`: `skip` (the default) reports the move `skipped` and continues with the rest of the plan; `abort` reports it `failed` and stops the *whole run* unconditionally — this one case ignores `execution.abort_on_failure` entirely, because the manual's own words for `on_timeout: abort` are "abort the run," a distinct, stronger promise than the general "stop after any failed move" policy a plain task failure still goes through.

### Failure and `abort_on_failure`

A failed `move_disk` task is reported `failed` with its detail; `apply` then lists any volume left behind on the target that is not referenced in the VM's config (`IMPLEMENTATION_PLAN.md` section 9.4) — **never deleted, only reported**, exactly like every other orphan this project surfaces. With `execution.abort_on_failure: true` (the default), the whole run stops there; with it `false`, `apply` continues with the plan's remaining

moves. The lock-timeout-abort case above is the one exception that always stops the run regardless of this setting.

### state.json: what a real run actually changes

apply is the one command that takes state.json's advisory lock (docs/internals/15-state.md) — checked first, before it ever touches PVE or Prometheus, so a second concurrent apply (any mode, including dry-run) exits 0 immediately and quietly rather than racing the first one. After the run, for every group where at least one move actually executed (moved or draining — both mean the mirror itself completed):

- last_balance.load_vector is replaced with that group's load as measured for this run, so the next run's drift gate compares against a real balance instead of treating every run as the first one.
- Every executed disk gets a fresh gates.cooldown_per_disk timestamp at its **new** location.
- **Both** of the move's storages — source and destination — get a fresh gates.cooldown_per_storage timestamp: section 6 says a storage "involved in a migration" accepts no new incoming moves, and section 9.3's sizing rule for this cooldown is specifically about protecting a *source* still draining a saferemove wipe, so recording the destination alone could never deliver on it. What the cooldown actually *excludes* on the next run stays destination-only (the heuristic/MILP backends refuse a cooldown storage as an incoming-move target, never as a source a disk may still leave) — recording both endpoints only widens which storages carry a timestamp, not what that timestamp blocks.

A group that never executes anything (the gate said NO ACTION, or a load_error/replan_needed stopped it before any move ran) leaves state.json untouched for that group.

### Crash and two-instance recovery

Before planning anything, every apply run (including dry-run) checks for a move_disk left running by a crashed previous run or a second, concurrently-running instance: state.json's own inflight_upids are re-checked, and every node's task list is scanned for a still-running qmmove task from this tool's own configured user. Any vmid this finds is excluded from this run's planning exactly like a manually-configured exclude.vmids entry — it shows up pinned in the report with the reason "excluded by config (exclude.vmids)", and a log line at WARNING explains which check actually found it. It is never force-cancelled, and this run never assumes it is safe to touch that VM's disk again.

During execution, state.json's inflight_upids is written **before** each move_disk is issued and cleared once that task itself has finished — synchronously, so a crash (killed, OOM, host reboot) partway through a move leaves a trace the *next* run's own startup check will find, rather than only ever being updated at the very end of a whole apply run. See docs/internals/93-crashrecovery.md for the full mechanism.

#### --json

Identical to plan's own groups[] shape (see docs/manual/27-plan.md), with one addition per group: "execution", either null (never reached — a load_error, NO ACTION, or a group the run never got to because an earlier one stopped it, e.g. an operator's [q]uit) or an object with stopped_early, stop_reason (null unless the run stopped for this group specifically) and outcomes[] — one entry per move actually attempted, each with disk_key, from_storage, to_storage, status (would_move/moved/skipped/failed/draining/replan_needed), detail, upid (null unless move_disk was actually issued for this move — always null for would_move/replan_needed, and for skipped except a lock timeout that happened after issuing nothing yet, so still null there too; a failed outcome carries one only when the task itself ran and failed, not when a lock timeout aborted before it started) and orphaned_volumes (only ever non-empty after a failed outcome).

## Safety properties, exit codes, and what this build actually does

### What is safe, unconditionally

These hold regardless of `execution.mode`, and are not configurable away (`.agents/domain-invariants.md`):

- **Dry-run is the default.** A missing or unparseable `execution.mode` means dry-run, never automatic execution.
- **The snapshot reserve is never traded for balance.** `pve-storage-drs` will leave a group imbalanced rather than breach `snapshot_reserve.factor`/`min_free_bytes` on any storage, and that reserve holds *during* a migration, not merely before and after it.
- **Nothing is ever deleted automatically.** An orphaned volume left by a failed move is reported, never removed. The operator deletes.
- **An explicitly named configuration that cannot be read or fails validation is a hard failure**, never a silent fallback to `/etc/pve/drs.yaml`.

### Exit codes

| Code | Meaning |
|---|---|
| `0` | Success. A run that stopped at a gate, found nothing worth moving, or (for `verify-metrics`) found no error-level finding. |
| `1` | The run failed: invalid configuration, an unreachable Prometheus or PVE API, a failed check, or a command not yet implemented in this build (see below). |
| `2` | Usage error on the command line. |

### What this build actually implements

`IMPLEMENTATION_PLAN.md` section 12 lays out nine phases. This is a work in progress; being honest about exactly where it stands matters more than a document that reads as if the tool were finished:

| Command | Status |
|---|---|
| `--version`, `--help`, `--manual` | Implemented. |
| `verify-metrics` | Implemented: all six `IMPLEMENTATION_PLAN.md` section 3.3 checks, human and `--json` output. |
| `show-load` | Implemented: every storage in every group, its disks (all buses), sizes, reserve status ((C4)/(C5)), per-disk/per-storage I/O load ($\ell$_d/L_s/u_s, section 4), and a section 6 act/no-act verdict per group with its reasoning, pinned and low-coverage disks flagged with their reason (including a disk within `gates.cooldown_per_disk`, once `state.json` has recorded one), human and `--json` output. A Prometheus outage degrades this one group's load (and its gate verdict) to "unavailable" rather than failing the whole command — sizes and reserve status are unaffected. The gate verdict now reads real drift history from `state.json` when a group has one recorded; a group with none yet still evaluates as a first run (reserve override or imbalance only, never drift-suppressed) — see `docs/internals/15-state.md` and `docs/internals/80-gates.md`. |
| `verify-storages` | Implemented: `saferemove` and the implied wipe time per storage, warning when `gates.cooldown_per_storage` or `migration.max_single_move_duration` is shorter than it. |

| Command | Status |
| --- | --- |
| `plan` | Implemented: per group, evaluates the section 6 gate (reading real drift history from `state.json` when a group has one recorded — same as `show-load`, see `docs/internals/15-state.md`), and on `ACT` solves it with `solver.backend` (`auto` tries CP-SAT, then CBC, then the dependency-free heuristic — see `docs/internals/91-optimize.md`; both MILP backends and the heuristic exclude, as a migration target, any storage within `gates.cooldown_per_storage`, except when repairing a live reserve violation, which is never deferred for a cooldown, in the heuristic only — see `docs/internals/91-optimize.md`'s own note on that gap), orders its moves under the section 8 transient reserve invariant, and checks the whole plan against section 7's payback (cost/benefit) test, including the section 7.3 saturation-ceiling guard (mirroring phase only) for any storage that configures `saturation_load` (still none in this project's own dogfooding cluster) — a plan resolving a reserve violation is exempt from the economic half of that test but not from the hard per-move duration rule or the saturation guard. Human and `--json` output report which backend actually solved each group. **No automatic re-solve-and-shrink on a failing payback test** — a plan that fails is reported, not silently adjusted (section 7.3's 3-retry loop is not implemented). See `docs/manual/27-plan.md` and `docs/internals/90-heuristic.md`/`91-optimize.md`/`95-schedule.md`/`96-payback.md`. |
| `explain` | Not implemented yet — needs the payback arithmetic it exists to explain (phase 5). |

| | |
|---|---|
| `apply` | Implemented for all three `execution.mode` values. Before planning anything, checks for a `move_disk` left running by a crashed previous run or a second concurrent instance (section 13) and excludes its vmid from this run; runs the identical per-group gate/solve/schedule/payback pipeline `plan` prints, then executes it: pre-flight re-checks each move against the live cluster immediately before issuing it (including exclusion tags/vmids), waits out a VM config lock rather than failing, and does not consider a move done until the task succeeds *and* the source volume is gone *and* the VM's lock is clear — a `saferemove` wipe can hold those apart for hours (`draining`, excluded as both source and target for the rest of that run). `state.json`'s `inflight_upids` is written before each `move_disk` and cleared once its task finishes, so a crash mid-move leaves a trace the next run's own startup check finds. `auto` additionally honours `execution.time_windows` (local host time; refuses a move that cannot finish before the window closes) and `execution.max_migrations_per_run` (a shared budget across every group the run visits), and re-invokes the whole pipeline from freshly observed state on a `replan_needed` mismatch, bounded by `execution.max_replans_per_run` (section 9.2's re-plan protocol). `execution.max_concurrent_migrations`/`max_concurrent_per_storage` above their default of `1` are honoured in `auto` mode: section 8.1's generalized transient invariant and section 9.2's "poll all in-flight UPIDs" loop, strictly FIFO (never reordering the scheduler's own queue to keep every slot busy) — `dry-run`/`confirm` always run strictly sequentially regardless. See `docs/manual/28-apply.md`, `docs/internals/92-execute.md` and `docs/internals/93-crashrecovery.md`. |

Configuration loading and validation (this whole manual's `10-configuration.md`) is complete and exercised by every command, including the ones above that are not yet implemented: a config error is reported and the run exits `1` before reaching the "not implemented" message, so validating a configuration file does not require waiting for the rest of the engine.

Running a command that is not implemented yet prints a message naming the plan section it belongs to, and exits `1`:

```
$ pve-storage-drs explain
pve-storage-drs: 'explain' is not implemented yet in this development build; see IMPLEMENTATION_PLAN.md
section 12 for the phase it belongs to
```

### Optional dependencies

`pve-storage-drs` runs, plans and executes (`dry-run`/`confirm`; `auto` not yet) with only `requests`, `ruamel.yaml` and `jsonschema` installed. Three dependencies are optional and are imported only where they are used, never at module level:

- **`pulp`** + **`coinor-cbc`** (Debian `Recommends`) — the packaged MILP solver path (`solver.backend: cbc`, or `auto` when CP-SAT is unavailable).
- **`ortools`** (`pip install proxmox-storage-drs[solver]`, **not** packaged for Debian — CP-SAT has no Debian package at all) — the preferred MILP backend (`solver.backend: cpsat`, or `auto`'s first choice) when installed by hand outside the Debian path.
- **`statsmodels`** (Debian `Suggests`) — needed only for `forecast.model: holt_winters`. Without it, that model logs a warning and falls back to `quantile` automatically rather than failing.

Without either MILP library, `solver.backend: auto` (the default) falls back to the dependency-free heuristic; nothing in `pve-storage-drs` requires a MILP solver to be installed at all — see `docs/internals/91-optimize.md`.